Security and Scalability Assessment

> *Prepared for: Investor Tech Audit Workshop Date: 2026-03-29 | Version: 1.0*

---

1. Authentication Architecture

Dual Auth Strategy

Platform	Method	Library	Token Storage
Web	NextAuth.js v4 (Credentials Provider)	`next-auth`	HTTP-only JWT cookie
Mobile	Custom JWT Bearer	`jsonwebtoken`	Expo SecureStore

Web Authentication Flow

```
1. User submits email + password at /auth/signin
2. NextAuth CredentialsProvider:
   a. Lookup User by email (Prisma)
   b. bcrypt.compare(password, user.passwordHash)
   c. Check user.status === 'approved' (approval gate)
   d. Return { id, email, name, role, status }
3. NextAuth creates JWT with custom claims (id, role, status)
4. JWT stored in HTTP-only session cookie
5. Subsequent requests: getServerSession(authOptions) extracts session
```

Mobile Authentication Flow

```
1. POST /api/mobile/login { email, password }
2. Server: bcrypt verify + status check
3. Server: jwt.sign({ userId, email, role }, NEXTAUTH_SECRET)
4. Client: stores token in Expo SecureStore
5. Subsequent requests: Authorization: Bearer <token>
6. Server: jwt.verify + extract userId
```

Registration Approval Gate

New user registration follows a mandatory admin approval workflow:

```
signup -> status: 'pending' -> admin review -> status: 'approved'|'rejected'
```

- Users with `status !== 'approved'` cannot sign in
- Auth error encodes status for client parsing: `"status:pending|optional_note"`
- Admin can add statusNote explaining rejection/suspension reason

---

2. Authorization

Role-Based Access Control

Role	Value	Scope
Learner	`role: 'learner'`	All user-facing features, limited by plan
Admin	`role: 'admin'`	Full system access including admin panel

Plan-Based Feature Gating

Feature	Basic (Free)	Standard	Advanced
Video recording length	5 seconds	30 seconds	3 minutes
AI Coach (Coach Ney)	--	Text only	Voice + Text
Arcade challenges	First 3 free	All	All + VIP
Foundation courses	All	All	All
Mini-games	All	All	All

Access Control Implementation

```
// src/services/access-control.ts
// Checks performed:
// 1. Authentication (session exists)
// 2. Approval status (user.status === 'approved')
// 3. Plan limits (video duration, coach access, etc.)
```

Admin Endpoint Protection

All `/api/admin/*` endpoints check:

```
const session = await getServerSession(authOptions)
if (!session) return 401
if ((session.user as any).role !== 'admin') return 403
```

---

3. Data Protection

Password Security

Aspect	Implementation
Hashing algorithm	bcryptjs
Salt rounds	Default (10)
Storage	`User.passwordHash` column
Plain text	Never stored, never logged

Token Security

Token Type	Generation	Storage	Expiry
JWT session	NextAuth auto	HTTP-only cookie	Session-based
Mobile JWT	`jsonwebtoken`	Expo SecureStore	Configurable
Verification tokens	NextAuth auto	Database	Time-limited
VAPID keys	Generated offline	Environment variables	No expiry

Sensitive Data Handling

Data	Protection
Passwords	bcrypt hashed, never returned in API responses
API keys	Environment variables only, never in code
Recording URLs	Vercel Blob signed URLs
Payment data	Delegated to PayPal/VNPay (no card data stored)
WhatsApp phones	Stored in LearnerProfile, requires explicit opt-in
Push endpoints	Encrypted via Web Push standard

Environment Variables

```
# Authentication
NEXTAUTH_SECRET          -- JWT signing secret
NEXTAUTH_URL             -- Canonical URL

# Database
DATABASE_URL             -- Neon connection string (pooled)
DIRECT_URL               -- Neon direct connection (migrations)

# AI Services
OPENAI_API_KEY           -- OpenAI API access

# Storage
BLOB_READ_WRITE_TOKEN    -- Vercel Blob access

# YouTube
YOUTUBE_API_KEY          -- Data API v3

# Payments
PAYPAL_CLIENT_ID         -- PayPal sandbox/live
PAYPAL_CLIENT_SECRET
VNPAY_TMN_CODE           -- VNPay terminal
VNPAY_HASH_SECRET

# Notifications
VAPID_PUBLIC_KEY         -- Web Push
```

```
VAPID_PRIVATE_KEY
RESEND_API_KEY           -- Email delivery
TWILIO_ACCOUNT_SID       -- WhatsApp
TWILIO_AUTH_TOKEN
TWILIO_WHATSAPP_FROM
```

---

4. API Security

Input Validation

Method	Usage
Zod	Schema validation on key endpoints (signup, clip creation, etc.)
TypeScript	Compile-time type safety across all route handlers
Prisma	Parameterized queries (SQL injection prevention)

Error Handling

- All API routes use try/catch with generic 500 responses
- Error details logged server-side, not exposed to clients
- Client error logging endpoint (`/api/logs`) with rate limiting

CORS

- Default Next.js CORS (same-origin unrestricted)
- Embed routes allow cross-origin for iframe mini-games
- Mobile app uses Bearer token (no CORS applicable to native)

Rate Limiting

Current state: Lightweight, endpoint-specific.

Endpoint	Current Limit	Recommended at Scale
`/api/logs`	Per-request check	10/min per IP
`/api/auth/signup`	None	5/min per IP
`/api/assistant/chat`	Plan-based (implicit)	Token bucket per user
`/api/public/games/*/submit`	None	60/min per session
All admin endpoints	Admin role required	IP allowlist

---

5. Scalability Architecture

Vercel Serverless Auto-Scaling

```
Request Volume    Vercel Function Instances
0-10 req/s    ->  1-3 instances (cold start possible)
10-100 req/s  ->  10-50 instances (warm)
```

```
100-1000 req/s -> 50-200 instances (auto-scaled)
1000+ req/s   ->  Enterprise plan required
```

Key characteristics:

- Each API route is an independent serverless function
- No shared in-memory state between invocations (stateless)
- Cold start: ~200-500ms (Next.js + Prisma client initialization)
- Warm instances: ~50-100ms response times

Neon PostgreSQL Scalability

Feature	Benefit
Connection pooling	PgBouncer built-in, handles 1000s of concurrent connections
Scale-to-zero	Compute pauses after inactivity (cost savings)
Auto-scaling compute	CPU/memory scales with query load
Read replicas	Available for read-heavy workloads
Branching	Database branches for development/testing

Prisma connection configuration:

```
datasource db {
  provider  = "postgresql"
  url       = env("DATABASE_URL")     // Pooled connection (PgBouncer)
  directUrl = env("DIRECT_URL")       // Direct connection (migrations)
}
```

Notification Queue Batching

The Learning Assistant Engine processes notifications in batches:

Parameter	Value	At Scale
Batch size	50 per invocation	Increase to 200+
Cron frequency	2x/day (08:00, 22:00 UTC)	Increase to hourly
Timezone bracketing	1-hour windows	Already scalable
Retry strategy	3 attempts, 15-min delay	Add exponential backoff

Cron Job Timezone Bracketing

```
08:00 UTC -> Process users in UTC+0 timezones (morning = 8 AM local)
09:00 UTC -> Process users in UTC-1 timezones
...
20:00 UTC -> Process users in UTC+12 timezones
```

At 2 cron invocations per day, the system processes 2 timezone brackets. At scale, increasing to hourly cron jobs (24/day) would cover all 24 timezone hours.

---

6. Cost Analysis

Estimated Monthly Costs by Scale

Infrastructure

Service	1K Users	10K Users	100K Users
Vercel (Pro plan)	$20	$20	$150+ (Enterprise)
Neon (Pro)	$19	$50	$200-500
Vercel Blob	$0 (included)	$10	$100-300
Domain + DNS	$15	$15	$15
Subtotal	**$54**	**$95**	**$465-965**

External Services (Usage-Based)

Service	1K Users (300 DAU)	10K Users (3K DAU)	100K Users (30K DAU)
OpenAI (Coach Ney + feedback)	$15	$150	$1,500
Resend (weekly emails)	$0 (free tier)	$20	$80
Twilio (WhatsApp, opt-in only)	$5	$50	$300
YouTube API (crawl jobs only)	$0 (quota)	$0	$0
Web Push (VAPID)	$0 (self-hosted)	$0	$0
Subtotal	**$20**	**$220**	**$1,880**

Total Estimated Monthly Cost

Scale	Total	Per User	Per Active User
1K users	**$74/mo**	$0.07	$0.25
10K users	**$315/mo**	$0.03	$0.11
100K users	**$2,345-2,845/mo**	$0.02-0.03	$0.08-0.09

Key cost insight: MediaPipe (client-side ML) eliminates what would be the largest cost -- server-side AI scoring. At 100K users with 30K DAU doing 5 recordings/day, that would be 150K GPT-4o Vision calls/day (~$4,500/day). MediaPipe reduces this to $0.

---

7. Performance Optimizations

Web Application

Optimization	Implementation
SSR + CSR hybrid	Next.js App Router with server components for static content, client components for interactive features
Edge caching	Vercel CDN caches static pages and assets globally
Image optimization	YouTube thumbnails served via YouTube CDN
Code splitting	Next.js automatic per-route code splitting
Lazy loading	MediaPipe models loaded on-demand (not at page load)
maxDuration	Extended timeout on AI-heavy routes (OpenAI calls)

Mobile Application

Optimization	Implementation
React.memo	Memoized components for list items (badges, quests, clips)
FlatList	Virtualized lists for large datasets (submissions, leaderboard)
SecureStore	Fast, encrypted token storage
API caching	Local state management with mock-data fallback
YouTube thumbnails	Native Image component with caching

Database

Optimization	Implementation
Connection pooling	Neon PgBouncer (DATABASE_URL)
Strategic indexes	62 indexes on high-query tables
Composite indexes	Multi-column indexes for common filter combinations
Cursor pagination	Used in batch operations (weekly report, queue processing)
Select optimization	Prisma `select` to fetch only needed fields
Parallel queries	`Promise.all` for independent queries in analyzers

---

8. Known Technical Debt

Item	Severity	Impact	Remediation
No Redis cache layer	Medium	DB hit on every API call	Add Redis for session/leaderboard caching
Lightweight rate limiting	Medium	Vulnerable to abuse at scale	Implement Upstash Redis rate limiter

No automated tests	High	Regression risk	Add Jest + Playwright test suites
Approximate timezone handling	Low	DST edge cases	Use `date-fns-tz` library
No database migrations	Medium	Schema changes via `db push`	Switch to `prisma migrate` for production
Payments in sandbox	Medium	No revenue collection	Complete PayPal/VNPay production integration
No GDPR compliance tools	Medium	EU market limitations	Add data export + deletion endpoints
Single-region database	Low	Latency for distant users	Neon multi-region or read replicas
No WAF	Medium	No DDoS protection beyond Vercel	Add Cloudflare or Vercel Firewall
Mobile auth token expiry	Low	Tokens don't expire	Add refresh token rotation

---

9. Security Recommendations for Production

Priority 1 (Before Launch)

- [ ] Add Redis-based rate limiting on auth and AI endpoints
- [ ] Implement CSRF protection on state-changing endpoints
- [ ] Add Content Security Policy headers
- [ ] Complete payment gateway production integration
- [ ] Add automated security scanning to CI pipeline

Priority 2 (First 90 Days)

- [ ] Implement API key rotation strategy for all external services
- [ ] Add audit logging for admin actions
- [ ] Implement data retention policies (GDPR Article 17)
- [ ] Set up monitoring and alerting (Vercel Analytics + custom)
- [ ] Conduct third-party security audit

Priority 3 (Scale Phase)

- [ ] Implement WAF (Cloudflare or Vercel Firewall)
- [ ] Add IP-based admin access restrictions
- [ ] Implement field-level encryption for sensitive data
- [ ] Add SOC 2 compliance documentation
- [ ] Set up bug bounty program

Document prepared by Data Engineer role. Security assessment based on codebase review as of 2026-03-29. No penetration testing was performed.